
Sound Loops, Widening, and Narrowing

Lecture 6 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

Completing the sound abstract interpreter

Widening and narrowing

§ 1

Completing the sound abstract interpreter

Available operators

For both the sign and interval abstractions, we have defined and proved sound:

- arithmetic and expression evaluation;
- assignment;
- boolean filters and abstract join.

Still to prove

- composition into a complete abstract semantics;
- soundness of `while` loops via least fixed-point approximation;
- soundness of the complete analyzer.

First task today

Complete the soundness proof shared by the sign and interval analyzers.

From collecting to abstract semantics

	Collecting semantics	Abstract semantics
<code>x := a</code>	$\text{assign}(x, a, S)$	$\text{assign}^\#(x, a, A)$
<code>c₁; c₂</code>	$\llbracket c_2 \rrbracket(\llbracket c_1 \rrbracket S)$	$\llbracket c_2 \rrbracket^\#(\llbracket c_1 \rrbracket^\# A)$
<code>if</code>	$\llbracket c_1 \rrbracket(\llbracket b \rrbracket S) \cup \llbracket c_2 \rrbracket(\llbracket \neg b \rrbracket S)$	$\llbracket c_1 \rrbracket^\#(\llbracket b \rrbracket^\# A) \sqcup \llbracket c_2 \rrbracket^\#(\llbracket \neg b \rrbracket^\# A)$
<code>while</code>	$\llbracket \neg b \rrbracket(\text{lfp} \Phi_S)$	$\llbracket \neg b \rrbracket^\#(\text{lfp} \Phi_A^\#)$

Concrete loop functional

$$\Phi_S(X) \triangleq S \cup \llbracket c \rrbracket(\llbracket b \rrbracket X).$$

Abstract loop functional

$$\Phi_A^\#(X) \triangleq A \sqcup \llbracket c \rrbracket^\#(\llbracket b \rrbracket^\# X).$$

The remaining proof obligation

Finite constructs compose sound operators. The `while` case also requires relating the two least fixed points.

Remark. In the interval analysis, we will not generally compute $\text{lfp} \Phi_A^\#$ exactly (see widening later).

The remaining case: while

Let $H_A^\sharp$ be the loop-head invariant returned by the abstract iterator, with $\Phi_A^\sharp(H_A^\sharp) \sqsubseteq^\sharp H_A^\sharp$. Define the abstract loop result as $\llbracket \neg b \rrbracket^\sharp(H_A^\sharp)$.

For command soundness, the loop case requires proving

$$\llbracket \text{while } b \text{ do } c \rrbracket(\gamma_\Sigma(A)) \subseteq \gamma_\Sigma(\llbracket \text{while } b \text{ do } c \rrbracket^\sharp A).$$

Unfolding the concrete loop semantics, the goal becomes

$$\llbracket \neg b \rrbracket(\text{lfp} \Phi_{\gamma_\Sigma(A)}) \subseteq \gamma_\Sigma(\llbracket \neg b \rrbracket^\sharp(H_A^\sharp)).$$

Monotonicity of $\llbracket \neg b \rrbracket$ and soundness of $\llbracket \neg b \rrbracket^\sharp$ reduce this goal to

$$\boxed{\text{lfp} \Phi_{\gamma_\Sigma(A)} \subseteq \gamma_\Sigma(H_A^\sharp).}$$

This is a *fixed-point transfer* problem.

Fixed-point transfer

Let L be a complete lattice, $f : L \rightarrow L$ monotone, $\gamma : L^\sharp \rightarrow L$ monotone, and $f^\sharp : L^\sharp \rightarrow L^\sharp$ sound: $f \circ \gamma \sqsubseteq \gamma \circ f^\sharp$. Then

$$f^\sharp(H^\sharp) \sqsubseteq^\sharp H^\sharp \implies \boxed{\text{lfp} f \sqsubseteq \gamma(H^\sharp)}.$$

Proof. Soundness and monotonicity of γ give

$$f(\gamma(H^\sharp)) \sqsubseteq \gamma(f^\sharp(H^\sharp)) \sqsubseteq \gamma(H^\sharp).$$

Thus $\gamma(H^\sharp)$ is an inductive invariant for f . By Park induction, $\text{lfp} f \sqsubseteq \gamma(H^\sharp)$.

Practical form

Any algorithm that returns a certified post-fixed point $H^\sharp$ suffices.

Applying fixed-point transfer to the loop

For every $X^\sharp$:

$$\begin{aligned}\Phi_{\gamma_\Sigma(A)}(\gamma_\Sigma(X^\sharp)) &= \gamma_\Sigma(A) \cup \llbracket c \rrbracket(\llbracket b \rrbracket(\gamma_\Sigma(X^\sharp))) \\ &\subseteq \gamma_\Sigma(A) \cup \llbracket c \rrbracket(\gamma_\Sigma(\llbracket b \rrbracket^\sharp X^\sharp)) \\ &\subseteq \gamma_\Sigma(A) \cup \gamma_\Sigma(\llbracket c \rrbracket^\sharp(\llbracket b \rrbracket^\sharp X^\sharp)) \\ &\subseteq \gamma_\Sigma(A \sqcup^\sharp \llbracket c \rrbracket^\sharp(\llbracket b \rrbracket^\sharp X^\sharp)) \\ &= \gamma_\Sigma(\Phi_A^\sharp(X^\sharp))\end{aligned}$$

by definition of Φ_S

by filter soundness and
monotonicity of $\llbracket c \rrbracket$

by soundness of the loop
body

by soundness of join

by definition of $\Phi_A^\sharp$.

Hence

$$\Phi_{\gamma_\Sigma(A)} \circ \gamma_\Sigma \sqsubseteq \gamma_\Sigma \circ \Phi_A^\sharp.$$

Therefore, for every $H_A^\sharp$ such that $\Phi_A^\sharp(H_A^\sharp) \sqsubseteq^\sharp H_A^\sharp$, fixed-point transfer yields

$$\boxed{\text{lfp} \Phi_{\gamma_\Sigma(A)} \subseteq \gamma_\Sigma(H_A^\sharp).}$$

Soundness of the loop

Let the abstract iterator return $H_A^\#$ satisfying

$$\Phi_A^\#(H_A^\#) \sqsubseteq^\# H_A^\#.$$

Fixed-point transfer gives

$$\text{lfp}\Phi_{\gamma_\Sigma(A)} \subseteq \gamma_\Sigma(H_A^\#).$$

Monotonicity of $\llbracket \neg b \rrbracket$ and soundness of $\llbracket \neg b \rrbracket^\#$ then yield

$$\llbracket \neg b \rrbracket(\text{lfp}\Phi_{\gamma_\Sigma(A)}) \subseteq \gamma_\Sigma(\llbracket \neg b \rrbracket^\#(H_A^\#)).$$

What the abstract iterator must guarantee

Compute in finite time a post-fixed point.

The sign analyzer computes the least fixed point

Abstract analysis of while

$$\llbracket \text{while } b \text{ do } c \rrbracket^\# A \triangleq \llbracket \neg b \rrbracket^\# (\text{lfp} \Phi_A^\#), \quad \Phi_A^\#(X) \triangleq A \sqcup^\# \llbracket c \rrbracket^\# (\llbracket b \rrbracket^\# X).$$

The abstract least fixed point is computed by Kleene iteration:

$$H_0 = \perp^\#, \quad H_{n+1} = \Phi_A^\#(H_n), \quad \text{lfp} \Phi_A^\# = H_N \quad \text{when } H_{N+1} = H_N.$$

Finite convergence

As seen in Lecture 3, this iteration terminates when $\Phi_A^\#$ is monotone and the abstract domain satisfies the ACC. The sign-state lattice is finite, hence it satisfies the ACC.

$\Phi_A^\#$ can be proved monotone by structural induction on commands, as for the collecting semantics in Lecture 3.

Soundness theorem

For every command c ,

$$\llbracket c \rrbracket \circ \gamma_{\Sigma} \subseteq \gamma_{\Sigma} \circ \llbracket c \rrbracket^{\#}.$$

If executing c from a state represented by A terminates, its final state is represented by $\llbracket c \rrbracket^{\#} A$.

Example

$$c \triangleq \text{if } x > 0 \text{ then } z := y + x \text{ else } z := 2 * y$$

The sign analysis computes

$$\llbracket c \rrbracket^{\#}[x \mapsto \top, y \mapsto +, z \mapsto \top] = [x \mapsto \top, y \mapsto +, z \mapsto +].$$

Thus, if initially $y > 0$, the final value of z is positive.

Exercise — a loop analyzed with signs

Consider the loop

```
while x <= 0 do x := x + 1
```

from the abstract entry state

$$A = [x \mapsto 0].$$

Its abstract loop functional is

$$\Phi_A^\#(H) \triangleq A \sqcup^\# \text{assign}^\#(x, x + 1, \llbracket x \leq 0 \rrbracket^\# H).$$

Task

Starting from $H_0 = \perp^\#$, compute the successive iterates $H_{n+1} = \Phi_A^\#(H_n)$ until stabilization, then apply the exit filter.

Solution — a loop analyzed with signs

$$H_{n+1} = [x \mapsto 0] \sqcup^{\#} \text{assign}^{\#}(x, x + 1, \llbracket x \leq 0 \rrbracket^{\#} H_n).$$

n	H_n	$\llbracket x \leq 0 \rrbracket^{\#} H_n$	$\llbracket x := x + 1 \rrbracket^{\#}(\cdot)$	H_{n+1}
0	$\perp^{\#}$	$\perp^{\#}$	$\perp^{\#}$	$[x \mapsto 0]$

Solution — a loop analyzed with signs

$$H_{n+1} = [x \mapsto 0] \sqcup^{\sharp} \text{assign}^{\sharp}(x, x + 1, \llbracket x \leq 0 \rrbracket^{\sharp} H_n).$$

n	H_n	$\llbracket x \leq 0 \rrbracket^{\sharp} H_n$	$\llbracket x := x + 1 \rrbracket^{\sharp}(\cdot)$	H_{n+1}
0	$\perp^{\sharp}$	$\perp^{\sharp}$	$\perp^{\sharp}$	$[x \mapsto 0]$
1	$[x \mapsto 0]$	$[x \mapsto 0]$	$[x \mapsto +]$	$[x \mapsto \top]$

Solution — a loop analyzed with signs

$$H_{n+1} = [x \mapsto 0] \sqcup^{\sharp} \text{assign}^{\sharp}(x, x + 1, \llbracket x \leq 0 \rrbracket^{\sharp} H_n).$$

n	H_n	$\llbracket x \leq 0 \rrbracket^{\sharp} H_n$	$\llbracket x := x + 1 \rrbracket^{\sharp}(\cdot)$	H_{n+1}
0	$\perp^{\sharp}$	$\perp^{\sharp}$	$\perp^{\sharp}$	$[x \mapsto 0]$
1	$[x \mapsto 0]$	$[x \mapsto 0]$	$[x \mapsto +]$	$[x \mapsto \top]$
2	$[x \mapsto \top]$	$[x \mapsto \top]$	$[x \mapsto \top]$	$[x \mapsto \top]$

The third line is stable. Applying the exit filter gives

$$\llbracket x > 0 \rrbracket^{\sharp} [x \mapsto \top] = [x \mapsto +].$$

Theorem 1

$(\mathbb{I}, \sqsubseteq_I)$ is a complete lattice.

Proof. Let $(I_i)_{i \in K}$ be a family of intervals. If every member is $\perp_I$ —including when $K = \emptyset$ —then

$$\bigsqcup_{i \in K}^I I_i = \perp_I.$$

Otherwise, let $K_+ \triangleq \{i \in K \mid I_i \neq \perp_I\}$. For every $i \in K_+$, write $I_i = [\ell_i, u_i]$. Then

$$\bigsqcup_{i \in K}^I I_i = \left[\inf_{i \in K_+} \ell_i, \sup_{i \in K_+} u_i \right].$$

Thus every family has a least upper bound.

Theorem 2

$(\mathbb{I}, \sqsubseteq_I)$ does not satisfy the ascending chain condition (ACC).

Task

1. Prove the theorem by giving an infinite strictly ascending chain in $\mathbb{I}$.
2. Give an infinite strictly descending chain in $\mathbb{I}$.

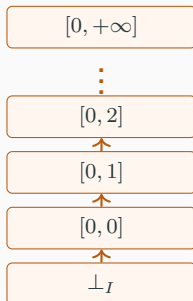
Exercise — infinite chains of intervals

Theorem 2

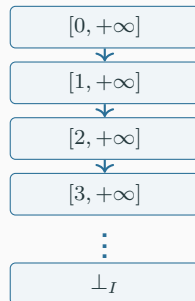
$(\mathbb{I}, \sqsubseteq_I)$ does not satisfy the ascending chain condition (ACC).

Solution

strictly ascending



strictly descending



The second chain also shows that $\mathbb{I}$ does not satisfy the descending chain condition (DCC).

The interval analyzer may not terminate

Analyze

```
x := 0; while true do x := x + 1
```

using plain Kleene iteration. At the loop head, it computes:

n	H_n	$\llbracket \text{true} \rrbracket^\# H_n$	$\llbracket x := x + 1 \rrbracket^\#(\cdot)$	$\Phi_A^\#(H_n)$
0	$\perp_I$	$\perp_I$	$\perp_I$	$[0, 0]$
1	$[0, 0]$	$[0, 0]$	$[1, 1]$	$[0, 1]$
2	$[0, 1]$	$[0, 1]$	$[1, 2]$	$[0, 2]$
3	$[0, 2]$	$[0, 2]$	$[1, 3]$	$[0, 3]$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

$$\perp_I \sqsubset_I [0, 0] \sqsubset_I [0, 1] \sqsubset_I [0, 2] \sqsubset_I \dots$$

Existence does not imply finite convergence

The preceding interval sequence never reaches a fixed point in finite time.

Mathematically

The least fixed point exists because intervals form a complete lattice. Here it is $[0, +\infty]$.

Algorithmically

Kleene iteration is not guaranteed to reach it because intervals do not satisfy the ACC.

Consequence

Completeness guarantees existence, not finite computation. We need to accelerate the ascending sequence.

§2

Widening and narrowing

Definition 3

A *widening* $\nabla : L \times L \rightarrow L$ satisfies:

1. $x \sqsubseteq x \nabla y$ and $y \sqsubseteq x \nabla y$;
2. for every sequence (X_n) , the sequence

$$I_0 = X_0, \quad I_{n+1} = I_n \nabla X_{n+1}$$

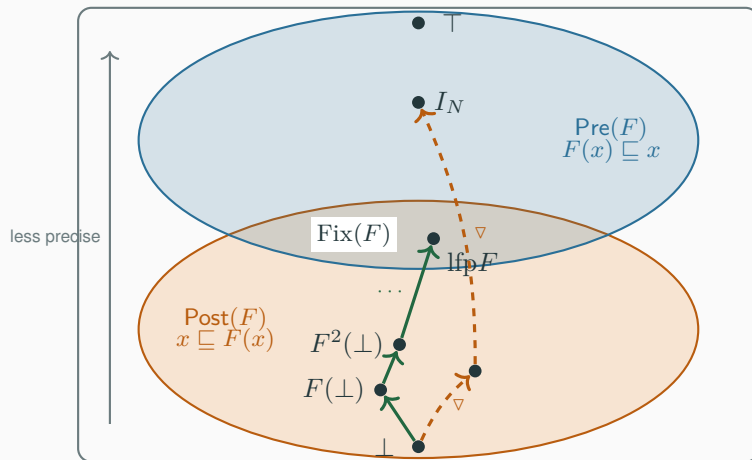
eventually stabilizes.

Replace the Kleene step by

$$I_{n+1} \triangleq I_n \nabla \Phi_A^\sharp(I_n).$$

- The sequence stabilizes at a post-fixed point $I_N: \Phi_A^\sharp(I_N) \sqsubseteq I_N$.
- When $\Phi_A^\sharp$ is monotone, Knaster–Tarski gives $\text{lfp} \Phi_A^\sharp \sqsubseteq I_N$: widening therefore computes an over-approximation of the least fixed point.

Kleene iteration and widening in the lattice



The standard widening on intervals

Let $X = [\ell_0, u_0]$ be the previous value and $Y = [\ell_1, u_1]$ the new candidate. Define

$$X \nabla_I Y \triangleq [\ell', u']$$

where

$$\ell' = \begin{cases} \ell_0 & \text{if } \ell_0 \leq \ell_1, \\ -\infty & \text{otherwise,} \end{cases} \quad u' = \begin{cases} u_0 & \text{if } u_1 \leq u_0, \\ +\infty & \text{otherwise.} \end{cases}$$

Also $\perp_I \nabla_I I = I$.

Theorem 4

The operator ∇_I is a widening on $(\mathbb{I}, \sqsubseteq_I)$.

The interval widening is a widening

Write $F \triangleq \Phi_A^\sharp$, $X = [\ell_0, u_0] = I_n$, and $Y = [\ell_1, u_1] = F(I_n)$.

1. The upper-bound conditions $X \sqsubseteq_I X \nabla_I Y$ and $Y \sqsubseteq_I X \nabla_I Y$ are visible in the four cases below.

(a) $\ell_0 \leq \ell_1$ and $u_1 \leq u_0$
 $I_{n+1} \equiv [\ell_0, u_0]$



(b) $\ell_0 \leq \ell_1$ and $u_0 < u_1$
 $I_{n+1} \equiv [\ell_0, +\infty]$



(c) $\ell_1 \leq \ell_0$ and $u_1 \leq u_0$
 $I_{n+1} \equiv [-\infty, u_0]$



(d) $\ell_1 \leq \ell_0$ and $u_0 < u_1$
 $I_{n+1} \equiv [-\infty, +\infty]$



2. Along a widened sequence, a finite lower bound is either kept or replaced once by $-\infty$; a finite upper bound is either kept or replaced once by $+\infty$. Hence the sequence stabilizes.

Intuition. Keep a stable bound; as soon as it moves outward, extrapolate it to infinity.

Exercise — widening intervals

Compute:

1. $[0, 0] \nabla_I [0, 1];$
2. $[-2, 4] \nabla_I [-3, 4];$
3. $[-2, 4] \nabla_I [-3, 7].$

Which bounds are remembered, and which are extrapolated?

Solution

$$\begin{aligned}[0, 0] \nabla_I [0, 1] &= [0, +\infty], \\ [-2, 4] \nabla_I [-3, 4] &= [-\infty, 4], \\ [-2, 4] \nabla_I [-3, 7] &= [-\infty, +\infty].\end{aligned}$$

A bound is kept exactly when the candidate does not move it outward.

Exercise — a loop with interval widening

Consider

`x := 0; while x <= 100 do x := x + 1.`

Writing only the interval for x , let

$$F(X) \triangleq [0, 0] \sqcup_I \text{assign}^\sharp(x, x + 1, \llbracket x \leq 100 \rrbracket^\sharp X),$$

and compute the widened sequence

$$I_0 = \perp_I, \quad I_{n+1} = I_n \nabla_I F(I_n).$$

Task

Compute the iterates until stabilization, then apply the exit filter. Compare the result with the exact final value.

Exercise — a loop with interval widening

Consider

`x := 0; while x <= 100 do x := x + 1.`

Writing only the interval for x , let

$$F(X) \triangleq [0, 0] \sqcup_I \text{assign}^\#(x, x + 1, \llbracket x \leq 100 \rrbracket^\# X),$$

and compute the widened sequence

$$I_0 = \perp_I, \quad I_{n+1} = I_n \nabla_I F(I_n).$$

Solution

$$I_1 = \perp_I \nabla_I [0, 0] = [0, 0],$$

$$I_2 = [0, 0] \nabla_I [0, 1] = [0, +\infty],$$

$$I_3 = [0, +\infty] \nabla_I [0, 101] = [0, +\infty].$$

The sequence is stable, and the exit filter gives

$$\llbracket x > 100 \rrbracket^\# [0, +\infty] = [101, +\infty].$$

The exact final value is 101.

Why widening remains sound

At convergence of the widened iteration,

$$I_N = I_N \nabla \Phi_A^\#(I_N).$$

Because widening returns an upper bound,

$$\Phi_A^\#(I_N) \sqsubseteq I_N \nabla \Phi_A^\#(I_N) = I_N.$$

Thus I_N is an inductive invariant, even when it is not the least fixed point.
Applying the fixed-point transfer theorem to I_N gives

$$\text{lfp}\Phi_{\gamma_\Sigma(A)} \subseteq \gamma_\Sigma(I_N).$$

Iterating from a post-fixed point

Let I_N be the post-fixed point obtained by widening. A first idea is to iterate F again:

$$D_0 \triangleq I_N, \quad D_{n+1} \triangleq F(D_n).$$

The first step improves precision:

$$D_1 = F(I_N) \sqsubseteq I_N = D_0.$$

Two difficulties

The transformer F need not be monotone when the loop body itself uses widening. Hence $D_{n+1} \sqsubseteq D_n$ need not imply $F(D_{n+1}) \sqsubseteq F(D_n)$. Moreover, descending iteration over intervals may fail to terminate.

What we need

A terminating refinement operator, together with an explicit check that every accepted iterate remains a post-fixed point.

Narrowing a post-fixed point

Let $D_0 \triangleq I_N$. At step n , the analyzer computes

$$C_n \triangleq F(D_n), \quad K_n \triangleq D_n \triangle C_n.$$

The analyzer then checks whether the candidate K_n is still a post-fixed point:

- if $F(K_n) \sqsubseteq K_n$, continue the iteration with $D_{n+1} \triangleq K_n$;
- otherwise, reject K_n and return the previous iterate D_n , which is known to be a post-fixed point.

Definition 5

A *narrowing* $\triangle : L \times L \rightarrow L$ satisfies, for all x, y ,

$$x \triangle y \sqsubseteq x,$$

and, for every sequence (X_n) , the sequence

$$D_0 = X_0, \quad D_{n+1} = D_n \triangle X_{n+1}.$$

eventually stabilizes.

Theorem 6

The checked sequence terminates at a post-fixed point D_M satisfying

$$F(D_M) \sqsubseteq D_M \sqsubseteq I_N.$$

Proof — narrowing preserves post-fixed points

We maintain the invariant

$$F(D_n) \sqsubseteq D_n \sqsubseteq I_N.$$

It holds initially because widening returned the post-fixed point I_N . Since narrowing is descending,

$$K_n = D_n \triangle F(D_n) \sqsubseteq D_n.$$

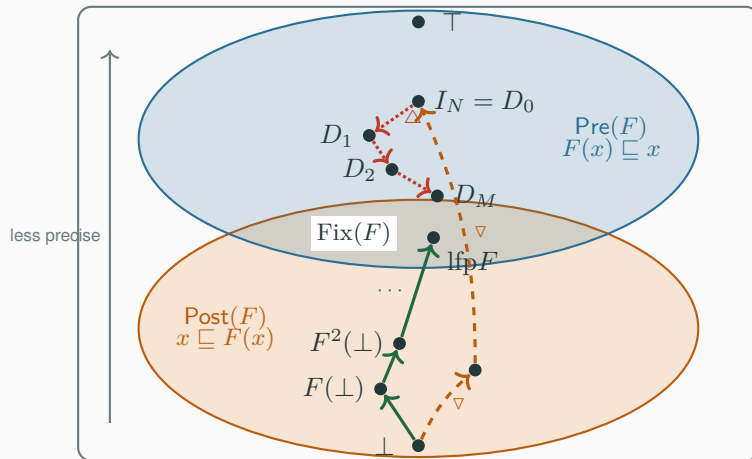
If $F(K_n) \sqsubseteq K_n$, accepting $D_{n+1} = K_n$ preserves both parts of the invariant. Otherwise, the analyzer stops and returns the previous D_n , which is still a post-fixed point below I_N .

If every candidate is accepted, the stabilization property of $\triangle$ applies to the sequence $(F(D_n))$ and forces some $D_{M+1} = D_M$. Therefore the analyzer always terminates and returns

$$F(D_M) \sqsubseteq D_M \sqsubseteq I_N.$$

A different definition of narrowing is sometimes used, but it assumes that F is monotone; this is not guaranteed here because nested loops may themselves use widening, and widening is not monotone.

Widening and narrowing in the lattice



The standard narrowing on intervals

Let $X = [\ell_0, u_0] = D_n$ be the current post-fixed point and $Y = [\ell_1, u_1] = F(D_n)$ the new candidate. Define

$$X \triangle_I Y \triangleq \begin{cases} [\ell', u'] & \text{if } Y \sqsubseteq_I X, \\ X & \text{otherwise,} \end{cases}$$

where

$$\ell' = \begin{cases} \ell_1 & \ell_0 = -\infty, \\ \ell_0 & \text{otherwise,} \end{cases} \quad u' = \begin{cases} u_1 & u_0 = +\infty, \\ u_0 & \text{otherwise.} \end{cases}$$

Also $\perp_I \triangle_I Y = \perp_I$.

Theorem 7

The operator $\triangle_I$ is a narrowing on $(\mathbb{I}, \sqsubseteq_I)$.

Intuition. Finite bounds are preserved, while each infinite bound can be replaced by a finite bound at most once; the sequence must therefore stabilize.

Exercise — a loop with interval narrowing

Consider again

`x := 0; while x <= 100 do x := x + 1,`

with

$$F(X) \triangleq [0, 0] \sqcup_I \text{assign}^\sharp(x, x + 1, \llbracket x \leq 100 \rrbracket^\sharp X).$$

Widening produced $D_0 = I_N = [0, +\infty]$.

Task

Compute the narrowing iterates until stabilization, then apply the exit filter.

Exercise — a loop with interval narrowing

Consider again

```
x := 0; while x <= 100 do x := x + 1,
```

with

$$F(X) \triangleq [0, 0] \sqcup_I \text{assign}^\sharp(x, x + 1, \llbracket x \leq 100 \rrbracket^\sharp X).$$

Widening produced $D_0 = I_N = [0, +\infty]$.

Solution

$$C_0 = F([0, +\infty]) = [0, 101],$$

$$K_0 = [0, +\infty] \triangle_I [0, 101] = [0, 101],$$

$$F(K_0) = [0, 101] \sqsubseteq_I K_0, \quad D_1 = [0, 101].$$

The next iterate is unchanged. The exit filter gives the exact final value

$$\llbracket x > 100 \rrbracket^\sharp [0, 101] = [101, 101],$$

The new loop-analysis algorithm

Ascending phase

$$\begin{aligned}C_n &\leftarrow \Phi_A^\#(I_n), \\I_{n+1} &\leftarrow I_n \nabla C_n.\end{aligned}$$

If $I_{n+1} = I_n$, set $D_0 \leftarrow I_n$ and start narrowing;
otherwise continue.

Descending phase

$$\begin{aligned}C_n &\leftarrow \Phi_A^\#(D_n), \\D_{n+1} &\leftarrow D_n \triangle C_n.\end{aligned}$$

Accept D_{n+1} only if $\Phi_A^\#(D_{n+1}) \sqsubseteq D_{n+1}$;
otherwise keep D_n and stop.

Exercise — interval analysis with widening and narrowing

Consider the program

```
x := 0; y := 10;  
while x < 10 do  
  x := x + 1;  
  y := y - 1
```

Task

Compute the result of the interval analysis at the loop head and at the loop exit, using widening followed by narrowing.

Present the widening and narrowing iterations in a table.

Solution — interval analysis with widening and narrowing

Write $\langle X, Y \rangle$ for the abstract environment in which $x \in X$ and $y \in Y$, and let $B = \llbracket x := x + 1; y := y - 1 \rrbracket^\#$.

phase	n	H_n	$B(\llbracket x < 10 \rrbracket^\# H_n)$	$F(H_n)$	next iterate
widening	0	$\perp_I$	$\perp_I$	$\langle [0, 0], [10, 10] \rangle$	$\langle [0, 0], [10, 10] \rangle$
widening	1	$\langle [0, 0], [10, 10] \rangle$	$\langle [1, 1], [9, 9] \rangle$	$\langle [0, 1], [9, 10] \rangle$	$\langle [0, +\infty], [-\infty, 10] \rangle$
widening	2	$\langle [0, +\infty], [-\infty, 10] \rangle$	$\langle [1, 10], [-\infty, 9] \rangle$	$\langle [0, 10], [-\infty, 10] \rangle$	$\langle [0, +\infty], [-\infty, 10] \rangle$
narrowing	0	$\langle [0, +\infty], [-\infty, 10] \rangle$	$\langle [1, 10], [-\infty, 9] \rangle$	$\langle [0, 10], [-\infty, 10] \rangle$	$\langle [0, 10], [-\infty, 10] \rangle$
narrowing	1	$\langle [0, 10], [-\infty, 10] \rangle$	$\langle [1, 10], [-\infty, 9] \rangle$	$\langle [0, 10], [-\infty, 10] \rangle$	$\langle [0, 10], [-\infty, 10] \rangle$

The loop-head invariant after narrowing is

$$x \in [0, 10], \quad y \in [-\infty, 10].$$

Applying the exit filter $x \geq 10$ gives

$x \in [10, 10], \quad y \in [-\infty, 10].$

Takeaways

1. Loop soundness follows from a sound post-fixed point and the fixed-point transfer theorem.
2. Infinite interval chains make plain Kleene iteration insufficient.
3. Widening forces termination; narrowing can recover precision while preserving soundness.